

EECE 320 – Project 2

Design and Simulation of a Simple Traffic Light Controller

Ali Zbeeb, Mohamad Hussein Karnib
Department of Electrical and Computer Engineering

July 24, 2026

Contents

1	Top-Level Design	2
1.1	Signal Definitions	2
1.2	Structural Verilog Implementation	3
2	Timing Counter	3
2.1	Counter Operation	3
2.2	Verilog Implementation	4
3	Signal Generator	4
3.1	State Encoding	4
3.2	The Two North–South Green States	5
3.3	State Transition Diagram	6
3.4	State Transition Table	6
3.5	Verilog Implementation	7
4	Verification of the Design Requirements	7
5	Testbench Design	8
5.1	Colour Field Decoding	8
5.2	Stimulus Sequence	8
5.3	Testbench Verilog	9
6	Simulation Results	10
6.1	Console Output	10
6.2	Waveform Output	12

1 Top-Level Design

The traffic light controller is organized as two sequential circuits that share a common clock **CK** and cooperate to drive the six traffic light outputs. The two units are:

- **Timing Counter** – a free-running modulo-60 counter that generates the internal timing signal **SW**. This signal defines the fundamental 50/10 second cadence of the Green–Yellow–Red sequence.
- **Signal Generator** – a finite state machine (FSM) that consumes **SW** together with the external input **SENSOR** and produces the six light outputs **NSRED**, **NSYEL**, **NSGRN**, **EWRED**, **EWYEL**, and **EWGRN**.

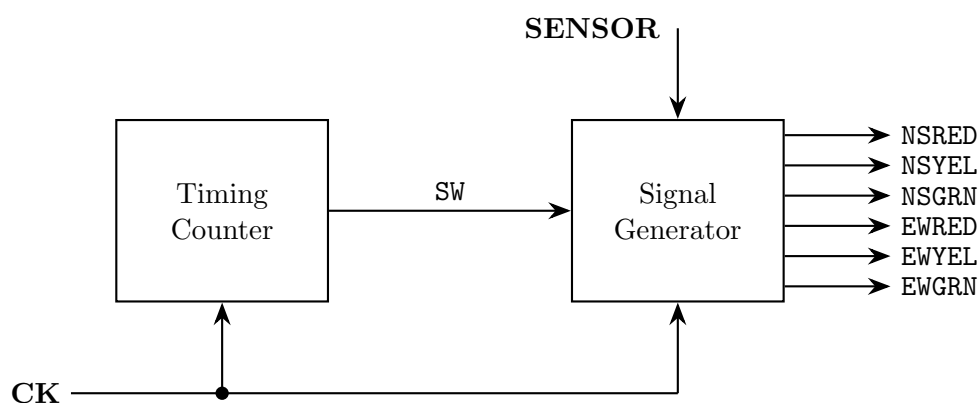


Figure 1: Top-level block diagram of the traffic light controller. The Timing Counter generates the internal timing signal **SW**, which drives the Signal Generator together with the external **SENSOR** input. Both units are clocked directly by **CK**.

1.1 Signal Definitions

Table 1 summarizes the function of every signal in the design.

Table 1: Input, internal, and output signals of the controller.

Signal	Type	Function
CK	Input	System clock, nominally one second period.
SENSOR	Input	Asserted while a car is waiting at the red light in the East–West direction.
SW	Internal	Timing signal from the counter; low for 50 cycles, high for 10 cycles.
NSRED	Output	Red light, north–south lane.
NSYEL	Output	Yellow light, north–south lane.
NSGRN	Output	Green light, north–south lane.
EWRED	Output	Red light, east–west lane.
EWYEL	Output	Yellow light, east–west lane.
EWGRN	Output	Green light, east–west lane.

1.2 Structural Verilog Implementation

Listing 1 shows the top-level module, which simply instantiates the two sub-units and wires SW between them. The signal SW is additionally brought out as a top-level port so that it can be observed directly in the waveform viewer during verification.

Listing 1: Top-level structural Verilog for the traffic controller.

```

1 //top level connects counter with generator
2 'include "timing_counter.sv"
3 'include "signal_generator.sv"
4
5 module traffic_controller (input CK, input SENSOR,
6                           output NSRED, output NSYEL, output NSGRN,
7                           output EWRED, output EWYEL, output EWGRN,
8                           output SW);
9
10    timing_counter    U_CNT (.CK(CK), .SW(SW));
11
12    signal_generator  U_GEN (.CK(CK), .SW(SW), .SENSOR(SENSOR),
13                            .NSRED(NSRED), .NSYEL(NSYEL), .NSGRN(NSGRN)
14                            ,
15                            .EWRED(EWRED), .EWYEL(EWYEL), .EWGRN(EWGRN)
16                            );
17
18 endmodule

```

2 Timing Counter

The Timing Counter is responsible for producing the fundamental timing signal SW. According to the specification, a full Green–Yellow–Red sequence for one direction spans $50 + 10 = 60$ clock cycles, so the counter is implemented as a free-running **modulo-60** counter.

2.1 Counter Operation

The counter holds a 6-bit register `count` that increments on every rising edge of CK and wraps from 59 back to 0. A 6-bit register is the minimum width capable of representing the range 0–59, since $2^6 = 64 \geq 60$.

The output SW is a purely combinational decode of the counter value:

$$SW = \begin{cases} 1, & 50 \leq \text{count} \leq 59 \\ 0, & 0 \leq \text{count} \leq 49 \end{cases} \quad (1)$$

Table 2: Phases of the modulo-60 timing counter.

count range	SW	Duration (cycles)
$0 \leq \text{count} \leq 49$	0	50
$50 \leq \text{count} \leq 59$	1	10
Total period		60

2.2 Verilog Implementation

Listing 2: Verilog implementation of the modulo-60 timing counter.

```

1 //mod 60 counter depends on cycles not real seconds.
2
3 module timing_counter (input CK, output SW);
4
5     reg [5:0] count = 6'd0;
6
7     always @(posedge CK) begin
8         if (count == 6'd59)
9             count <= 6'd0;
10        else
11            count <= count + 6'd1;
12    end
13
14    assign SW = (count >= 6'd50);
15
16 endmodule

```

3 Signal Generator

The Signal Generator is a Moore finite state machine. Its outputs depend only on the current state, which makes the six light signals glitch-free and unambiguous: in every state exactly one light is illuminated per direction.

3.1 State Encoding

The FSM uses five states, encoded in a 3-bit state register. The two distinct north–south green states are the key design decision of this project, and are explained in Section 3.2.

Table 3: State encoding and output assignment of the Signal Generator.

State	Encoding	NSGRN	NSYEL	NSRED	EWGRN	EWYEL	EWRED
NSG0	3'd0	1	0	0	0	0	1
NSG1	3'd1	1	0	0	0	0	1
NSY	3'd2	0	1	0	0	0	1
EWG	3'd3	0	0	1	1	0	0
EWY	3'd4	0	0	1	0	1	0

The output equations follow directly from Table 3:

$$\text{NSGRN} = (\text{state} = \text{NSGO}) + (\text{state} = \text{NSG1}) \quad (2)$$

$$\text{NSYEL} = (\text{state} = \text{NSY}) \quad (3)$$

$$\text{NSRED} = (\text{state} = \text{EWG}) + (\text{state} = \text{EWY}) \quad (4)$$

$$\text{EWGRN} = (\text{state} = \text{EWG}) \quad (5)$$

$$\text{EWYEL} = (\text{state} = \text{EWY}) \quad (6)$$

$$\text{EWRED} = (\text{state} = \text{NSGO}) + (\text{state} = \text{NSG1}) + (\text{state} = \text{NSY}) \quad (7)$$

3.2 The Two North–South Green States

A basic design would use a single north–south green state. That design fails the specification requirement that *no yellow period may be shortened*, and the reason is worth stating precisely.

The yellow interval must last exactly 10 cycles, and it must be terminated by the falling edge of **SW** (the moment **count** wraps from 59 to 0). Therefore the FSM may only *enter* the yellow state at a moment when **SW** has just risen, i.e. at the start of a full 10-cycle **SW**-high pulse. If the FSM were permitted to enter **NSY** at an arbitrary time, for example when **SENSOR** is asserted three cycles into an **SW**-high pulse, then the yellow light would only remain lit for the remaining seven cycles before **SW** fell and forced the transition to **EWG**. This is precisely the failure mode the specification warns about.

The solution is to split the north–south green condition into two states that differ only in the value of **SW** that produced them:

- **NSGO** : north–south green, entered and held while **SW** is *low*. This is the only state from which the FSM is permitted to sample **SENSOR** and depart.
- **NSG1** : north–south green, entered when the machine was in **NSGO** at the rise of **SW** with no car waiting. This is the “stalled” state, which absorbs the entire **SW**-high pulse and returns to **NSGO** when **SW** falls.

Because the transition **NSGO** $\rightarrow$ **NSY** is qualified by **SW** = 1, and because **NSGO** is only ever occupied while **SW** = 0, that transition can only fire on the *rising edge* of **SW**. The yellow state is therefore always entered at the beginning of a complete **SW**-high pulse and always receives its full 10 cycles.

3.3 State Transition Diagram

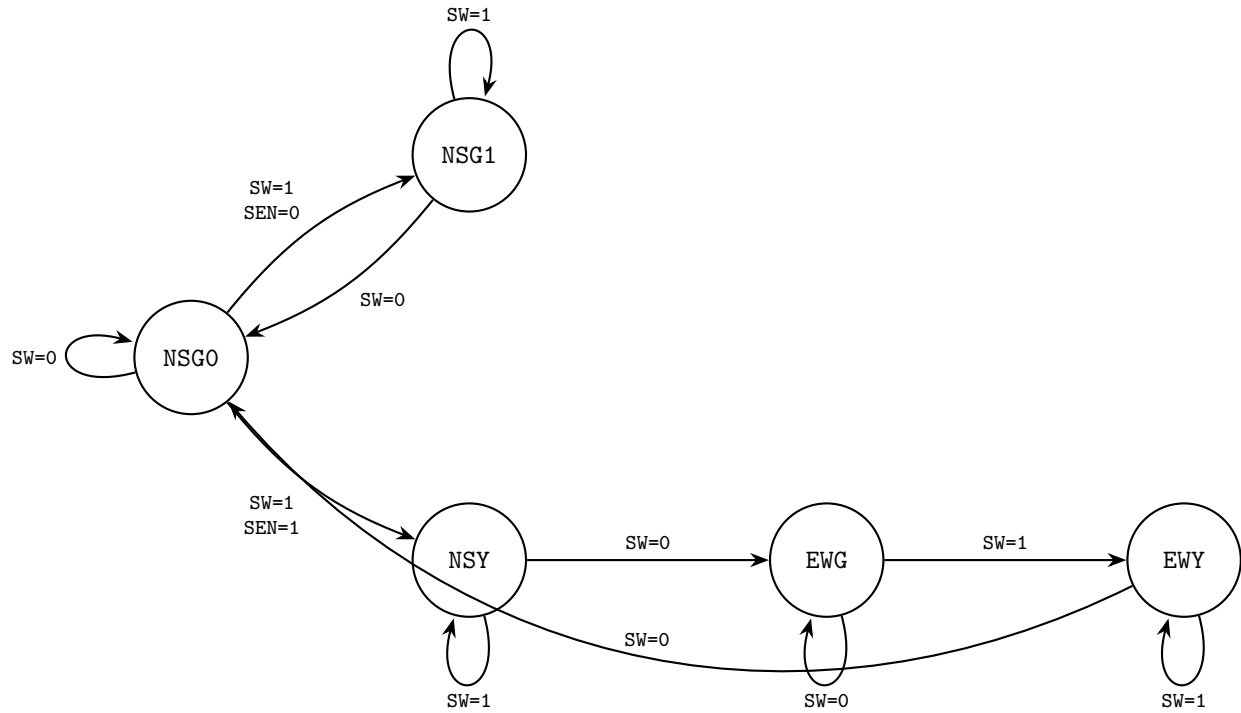


Figure 2: State transition diagram of the Signal Generator. Only the transition $\text{NSG0} \rightarrow \text{NSY}$ depends on **SENSOR**; every other transition is driven solely by **SW**, which guarantees that each interval lasts its full duration.

3.4 State Transition Table

Table 4 gives the complete next-state function. A dash indicates a don't-care condition on **SENSOR**.

Table 4: Next-state table for the Signal Generator.

Current State	SW	SENSOR	Next State	
NSG0	0	–	NSG0	(hold green, wait for SW)
NSG0	1	0	NSG1	(no car: stall)
NSG0	1	1	NSY	(car waiting: begin yellow)
NSG1	1	–	NSG1	(absorb the SW pulse)
NSG1	0	–	NSG0	
NSY	1	–	NSY	(full 10-cycle yellow)
NSY	0	–	EWG	
EWG	0	–	EWG	(full 50-cycle green)
EWG	1	–	EWY	
EWY	1	–	EWY	(full 10-cycle yellow)
EWY	0	–	NSG0	

3.5 Verilog Implementation

Listing 3: Verilog implementation of the Signal Generator FSM.

```

1  module signal_generator (input CK, input SW, input SENSOR,
2                          output NSRED, output NSYEL, output NSGRN,
3                          output EWRED, output EWYEL, output EWGRN);
4
5      localparam [2:0] NSG0 = 3'd0,    // NS green when sw is low
6                          NSG1 = 3'd1,    // NS green stalled
7                          NSY  = 3'd2,    // NS yellow
8                          EWG  = 3'd3,    // EW green
9                          EWY  = 3'd4;    // EW yellow
10
11     reg [2:0] state = NSG0;
12     reg [2:0] next_state;
13
14     // next state logic
15     always @(*) begin
16         case (state)
17             NSG0:    next_state = (SW == 1'b0) ? NSG0 :
18                                (SENSOR) ? NSY : NSG1;
19             NSG1:    next_state = (SW == 1'b1) ? NSG1 : NSG0;
20             NSY :    next_state = (SW == 1'b1) ? NSY : EWG;
21             EWG :    next_state = (SW == 1'b0) ? EWG : EWY;
22             EWY :    next_state = (SW == 1'b1) ? EWY : NSG0;
23             default: next_state = NSG0;
24         endcase
25     end
26
27     // state register controls all flip flops with the same clock
28     always @(posedge CK)
29         state <= next_state;
30
31     assign NSGRN = (state == NSG0) | (state == NSG1);
32     assign NSYEL = (state == NSY);
33     assign NSRED = (state == EWG) | (state == EWY);
34
35     assign EWGRN = (state == EWG);
36     assign EWYEL = (state == EWY);
37     assign EWRED = (state == NSG0) | (state == NSG1) | (state == NSY);
38
39 endmodule

```

4 Verification of the Design Requirements

Before presenting the simulation results, this section states how each explicit requirement of the specification is satisfied by the design.

Table 5: Mapping of specification requirements to design features.

Requirement	How it is satisfied
SW must not clock any flip-flop	The only flip-flops are the 6-bit counter and the 3-bit state register, both triggered by posedge CK. SW appears only in combinational expressions.
120 clock cycles per full Green–Yellow–Red sequence	Each direction occupies one 60-cycle SW period (50+10); two directions give $2 \times 60 = 120$ cycles.
Counter-based timing unit	Implemented as a free-running modulo-60 counter (Listing 2).
Signal generator as a regular FSM	Five-state Moore machine with a two-process coding style (Listing 3).
North–south favored when no car is waiting	The $NSGO \leftrightarrow NSG1$ cycle holds $NSGRN$ indefinitely while $SENSOR$ is low.
No interval shortened by a mid-period $SENSOR$ change	$SENSOR$ is sampled only in $NSGO$ and only when SW rises; all other transitions ignore $SENSOR$ entirely.
Full yellow period after a stall	Entry into NSY coincides with the rising edge of SW, so yellow always spans the complete 10-cycle pulse.

5 Testbench Design

The testbench instantiates the complete controller, drives CK with a 10 ns period, and applies a $SENSOR$ stimulus sequence designed to exercise both test cases required by the specification.

5.1 Colour Field Decoding

As required by the deliverables, the testbench derives two 24-bit string fields, NS and EW , which display the active colour of each direction as readable text in the console and in the waveform viewer:

$$NS = NSGRN ? \text{“GRN”} : NSYEL ? \text{“YEL”} : NSRED ? \text{“RED”} : \text{“???”} \quad (8)$$

$$EW = EWGRN ? \text{“GRN”} : EWYEL ? \text{“YEL”} : EWRED ? \text{“RED”} : \text{“???”} \quad (9)$$

5.2 Stimulus Sequence

The stimulus is organized into four phases, summarized in Table 6.

Table 6: Phases of the testbench stimulus sequence.

Phase	SENSOR	Purpose
1	0	Idle for 130 cycles. Verifies that north–south remains green indefinitely with no car waiting.
2	0 → 1	Test 1: assert SENSOR while SW is low, then run 250 cycles. Verifies the normal alternating sequence.
3	1 → 0	Deassert during north–south green. Verifies that the controller stalls again.
4	0 → 1	Test 2: assert SENSOR three cycles <i>after</i> the rising edge of SW . Verifies that the yellow period is not shortened.

5.3 Testbench Verilog

Listing 4: Testbench for the complete traffic light controller.

```

1  `timescale 1ns/1ps
2
3  module traffic_tb;
4
5      reg  CK = 1'b0;
6      reg  SENSOR = 1'b0;
7      wire NSRED, NSYEL, NSGRN, EWRED, EWYEL, EWGRN, SW;
8
9      traffic_controller UUT (.CK(CK), .SENSOR(SENSOR),
10                             .NSRED(NSRED), .NSYEL(NSYEL), .NSGRN(NSGRN)
11                             ,
12                             .EWRED(EWRED), .EWYEL(EWYEL), .EWGRN(EWGRN)
13                             ,
14                             .SW(SW));
15
16      always #5 CK = ~CK;
17
18      //initialize colors
19      wire [23:0] NS = NSGRN ? "GRN" : NSYEL ? "YEL" : NSRED ? "RED" :
20                      "???";
21      wire [23:0] EW = EWGRN ? "GRN" : EWYEL ? "YEL" : EWRED ? "RED" :
22                      "???";
23
24      initial begin
25          $dumpfile("dump.vcd");
26          $dumpvars(0, traffic_tb);
27          $monitor("T=%0t SW=%b SEN=%b NS=%s EW=%s",
28                  $time, SW, SENSOR, NS, EW);
29
30          //no cars on eastwest
31          repeat (130) @(posedge CK);
32
33          // test 1

```

```

30     wait (SW == 1'b0);
31     repeat (20) @(posedge CK);
32     SENSOR = 1'b1;
33     $display("TEST 1: SENSOR asserted at T=%0t (SW=%b)", $time, SW)
34     ;
35     repeat (250) @(posedge CK);
36
37     // drop the sensor
38     wait (EWYEL == 1'b1);           // end of an EW
39     wait (NSGRN == 1'b1);           // NS green started
40     repeat (2) @(posedge CK);
41     SENSOR = 1'b0;
42     $display("SENSOR deasserted at T=%0t; controller should stall",
43             $time);
44     repeat (130) @(posedge CK);
45
46     // test 2
47     @(posedge SW);
48     repeat (3) @(posedge CK);
49     SENSOR = 1'b1;
50     $display(" TEST 2: SENSOR asserted MID-PULSE at T=%0t (SW=%b)",
51             $time, SW);
52
53     //ns green then yellow
54     repeat (200) @(posedge CK);
55
56     $finish;
57 end
58
59 endmodule

```

6 Simulation Results

6.1 Console Output

Figure 3 shows the console transcript produced by the `$monitor` statement. Each line reports the simulation time, the value of `SW`, the value of `SENSOR`, and the decoded colours of both directions.

```

# KERNEL: T=595000 SW=0 SEN=0 NS=GRN EW=RED
# KERNEL: T=1095000 SW=1 SEN=0 NS=GRN EW=RED
# KERNEL: T=1195000 SW=0 SEN=0 NS=GRN EW=RED
# KERNEL: TEST 1: SENSOR asserted at T=1495000 (SW=0)
# KERNEL: T=1495000 SW=0 SEN=1 NS=GRN EW=RED
# KERNEL: T=1695000 SW=1 SEN=1 NS=GRN EW=RED
# KERNEL: T=1705000 SW=1 SEN=1 NS=YEL EW=RED
# KERNEL: T=1795000 SW=0 SEN=1 NS=YEL EW=RED
# KERNEL: T=1805000 SW=0 SEN=1 NS=RED EW=GRN
# KERNEL: T=2295000 SW=1 SEN=1 NS=RED EW=GRN
# KERNEL: T=2305000 SW=1 SEN=1 NS=RED EW=YEL
# KERNEL: T=2395000 SW=0 SEN=1 NS=RED EW=YEL
# KERNEL: T=2405000 SW=0 SEN=1 NS=GRN EW=RED
# KERNEL: T=2895000 SW=1 SEN=1 NS=GRN EW=RED
# KERNEL: T=2905000 SW=1 SEN=1 NS=YEL EW=RED
# KERNEL: T=2995000 SW=0 SEN=1 NS=YEL EW=RED
# KERNEL: T=3005000 SW=0 SEN=1 NS=RED EW=GRN
# KERNEL: T=3495000 SW=1 SEN=1 NS=RED EW=GRN
# KERNEL: T=3505000 SW=1 SEN=1 NS=RED EW=YEL
# KERNEL: T=3595000 SW=0 SEN=1 NS=RED EW=YEL
# KERNEL: T=3605000 SW=0 SEN=1 NS=GRN EW=RED
# KERNEL: T=4095000 SW=1 SEN=1 NS=GRN EW=RED
# KERNEL: T=4105000 SW=1 SEN=1 NS=YEL EW=RED
# KERNEL: T=4195000 SW=0 SEN=1 NS=YEL EW=RED
# KERNEL: T=4205000 SW=0 SEN=1 NS=RED EW=GRN
# KERNEL: T=4695000 SW=1 SEN=1 NS=RED EW=GRN
# KERNEL: T=4705000 SW=1 SEN=1 NS=RED EW=YEL
# KERNEL: T=4795000 SW=0 SEN=1 NS=RED EW=YEL
# KERNEL: T=4805000 SW=0 SEN=1 NS=GRN EW=RED
# KERNEL: SENSOR deasserted at T=4825000; controller should stop!
# KERNEL: T=4825000 SW=0 SEN=0 NS=GRN EW=RED
# KERNEL: T=5295000 SW=1 SEN=0 NS=GRN EW=RED
# KERNEL: T=5395000 SW=0 SEN=0 NS=GRN EW=RED
# KERNEL: T=5895000 SW=1 SEN=0 NS=GRN EW=RED
# KERNEL: T=5995000 SW=0 SEN=0 NS=GRN EW=RED
# KERNEL: T=6495000 SW=1 SEN=0 NS=GRN EW=RED
# KERNEL: TEST 2: SENSOR asserted MID-PULSE at T=6525000 (SW=1)
# KERNEL: T=6525000 SW=1 SEN=1 NS=GRN EW=RED
# KERNEL: T=6595000 SW=0 SEN=1 NS=GRN EW=RED
# KERNEL: T=7095000 SW=1 SEN=1 NS=GRN EW=RED
# KERNEL: T=7105000 SW=1 SEN=1 NS=YEL EW=RED
# KERNEL: T=7195000 SW=0 SEN=1 NS=YEL EW=RED
# KERNEL: T=7205000 SW=0 SEN=1 NS=RED EW=GRN
# KERNEL: T=7695000 SW=1 SEN=1 NS=RED EW=GRN

```

Figure 3: Simulation console output showing the decoded NS and EW colour fields, with the two test markers annotated by $\$display$ statements.

6.2 Waveform Output

Figure 4 shows the EPWave viewer output for the alternating phase of the simulation. The two string buses NS[23:0] and EW[23:0] are displayed alongside the six individual light signals.

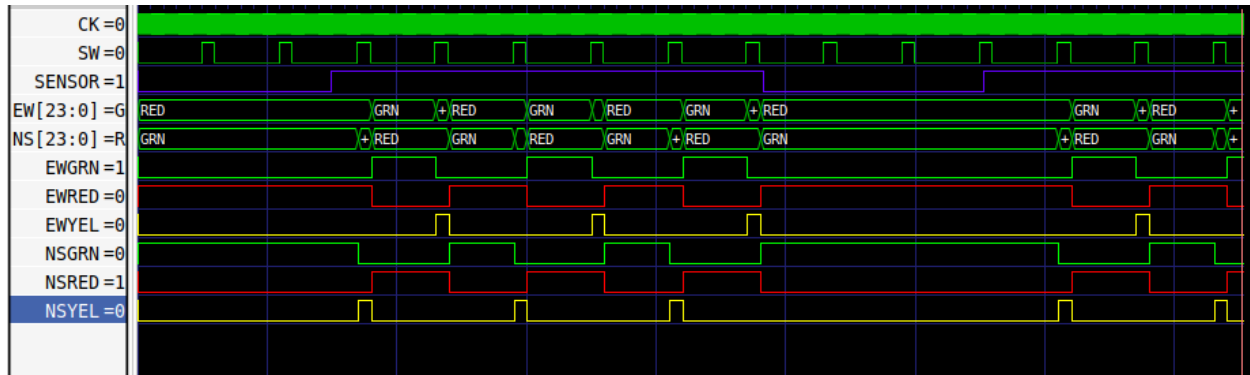


Figure 4: EPWave simulation waveform. The NS and EW buses show the decoded colour of each direction, and the six light outputs below confirm that exactly one lamp per direction is asserted at any instant.